Lecture 14: RNN++

Instructor: LI Yu Scribe: JIANG Huanyun 4 November 2026

1 Back-propagation through time

• Hidden state model: $h_t = f(x_t, h_{t-1})$ computes the state, $o_t = g(h_t, w_o)$ the output. Classification uses the last hidden state; regression may predict at every position; any model conditioned on previous values is autoregressive.

• Training with GD: initialize all weights randomly, run the forward pass over all training data to compute Y, then update $w_{ij}^{(k)} \leftarrow w_{ij}^{(k)} - \alpha \cdot \partial \mathrm{Div}(Y, d)/\partial w_{ij}^{(k)}$ and repeat until convergence.

• Chain rule: $dy/dx = (df/dg)(dg/dx)$ for nested functions; for a distributed nesting $y = f(g_1(x), \ldots, g_M(x))$, *the total derivative is the sum of the contributions through each path,* $dy/dx = \Sigma\, dy/dz_j \cdot dz_j/dx$.

• RNNs are trained with back-propagation and standard optimizers, but the recurrent dependency makes the gradient computation subtle.

1.1 Gradient flow through the recurrent cell

• With $h_t = f(x_t, h_{t-1}, w_h)$ and $o_t = g(h_t, w_o)$, the loss $L = (1/T)\Sigma\, t_t l(y_t, o_t)$ sums over all T steps, so gradients of w_h flow through every step:

$$\frac{\partial L}{\partial w_h} = \sum_{t=1}^{T} \frac{1}{T} \cdot \frac{\partial l(y_t, o_t)}{\partial o_t} \cdot \frac{\partial g(h_t, w_o)}{\partial h_t} \cdot \frac{\partial h_t}{\partial w_h}$$

• The last factor expands recursively; with the linear cell $h_t = \mathbf{W}_{hx} x_t + \mathbf{W}_{hh} h_{t-1}$, $o_t = \mathbf{W}_{qh} h_t$, the terms are powers of $\mathbf{W}_{hh}$ raised along the chain:

$$\frac{\partial L}{\partial h_t} = \sum_{i=t}^{T} \mathbf{W}_{hh}^{T-i} \mathbf{W}_{qh}^{T} \cdot \frac{\partial L}{\partial o_{T+t-i}}$$

• For a large spectral radius the product explodes, for a small one it vanishes. Long chains are slow, may explode or vanish — this motivates truncation.

1.2 Truncation

• Full back-propagation through all T steps is slow and numerically fragile; gradients can explode or vanish.

• Truncating time steps: stop the sum after τ steps so the longest dependency is at most τ.

• Randomized truncation: stop after a random number of steps, which biases the estimate while stabilizing it.

2 Gated recurrent units (GRU)

• Prior knowledge: text can be long; early observations may be important (a naive cell may decay them away); some information is meaningless and should be skipped; a logic break should reset the state.

• The plain recurrence $H_t = \varphi(X_t \mathbf{W}_{xh} + H_{t-1} \mathbf{W}_{hh} + b_h)$ always mixes everything; we need gates that turn information on or off to memorize, skip, and reset.

2.1 Building the gates

• The four attempts below add controls one by one: reset gates turn off parts of H_{t-1}, input gates control X_t, pass-through terms keep the state, and the final GRU update interpolates between the candidate and the previous state.

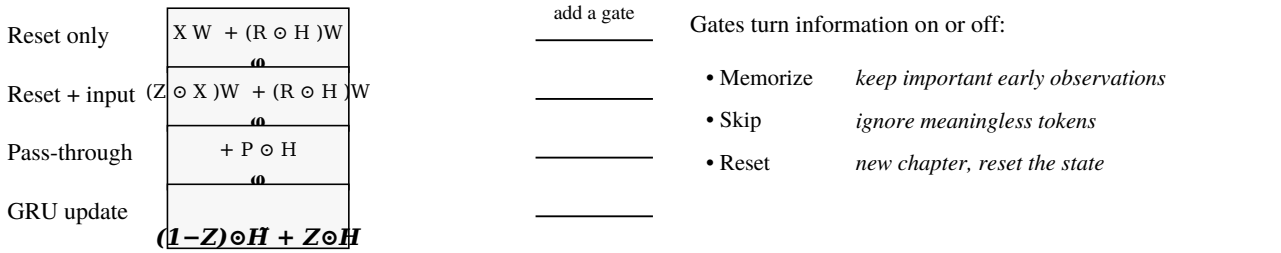


Figure 1. The plain recurrence gains gates one by one: reset, input, pass-through, and finally the GRU interpolation between a candidate state and the previous state.

• First attempt (reset only): $H_t = \varphi(X_t \mathbf{W}_{xh} + (R_t \odot H_{t-1}) \mathbf{W}_{hh} + b_h)$ with a binary vector R_t that zeroes out some previous states. It can forget, but cannot skip the current input.

• Second attempt (reset + input gate): $H_t = \varphi((Z_t \odot X_t) \mathbf{W}_{xh} + (R_t \odot H_{t-1}) \mathbf{W}_{hh} + b_h)$. If both gates close, the cell collapses to $\varphi(b_h)$ — information is lost.

• Third attempt: add a pass-through term $P_t \odot H_{t-1}$. Memorizing must be exclusive from resetting, and the pass-through should be complementary to skipping.

• Fourth attempt (the GRU update): replace the separate controls with one interpolation between a candidate state and the previous state.

2.2 GRU equations

• Reset gate R_t decides how much of H_{t-1} enters the candidate; update gate Z_t interpolates between the candidate and H_{t-1}. Gate outputs are near 0 or 1 from the sigmoid σ, while the candidate uses *tanh* .

$$R_t = \sigma(X_t W_{xr} + H_{t-1} W_{hr} + b_r), \quad Z_t = \sigma(X_t W_{xz} + H_{t-1} W_{hz} + b_z)$$

$$\tilde{H}_t = \tanh(X_t W_{xh} + (R_t \odot H_{t-1}) W_{hh} + b_h),$$

$$H_t = (1 - Z_t) \odot \tilde{H}_t + Z_t \odot H_{t-1}.$$

• Reset gates help capture short-term dependencies in sequences; update gates help capture long-term dependencies in sequences.

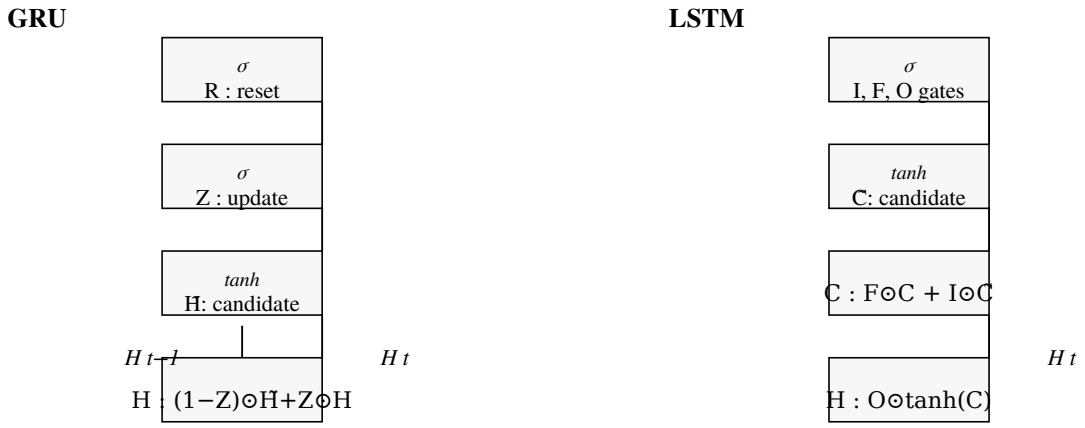


Figure 2. GRU uses two gates to interpolate between a candidate state and the previous state; LSTM keeps a dedicated cell state that is updated additively through forget and input gates.

3 Long short-term memory (LSTM)

• Memory is important: could we have a dedicated memory cell? The answer combines three gates — input, forget, and output — with a cell state that flows through time.

• The cell state C_t is updated additively, so gradients can flow without repeated multiplication; this alleviates vanishing and exploding gradients.

$$\mathbf{I}_t = \sigma(\mathbf{X}_t \mathbf{W}_{xi} + \mathbf{H}_{t-1} \mathbf{W}_{hi} + \mathbf{b}_i), \quad \mathbf{F}_t = \sigma(\mathbf{X}_t \mathbf{W}_{xf} + \mathbf{H}_{t-1} \mathbf{W}_{hf} + \mathbf{b}_f)$$

$$\mathbf{O}_t = \sigma(\mathbf{X}_t \mathbf{W}_{xo} + \mathbf{H}_{t-1} \mathbf{W}_{ho} + \mathbf{b}_o),$$

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{X}_t \mathbf{W}_{xc} + \mathbf{H}_{t-1} \mathbf{W}_{hc} + \mathbf{b}_c)$$

$$\mathbf{C}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t,$$

$$\mathbf{H}_t = \mathbf{O}_t \odot \tanh(\mathbf{C}_t).$$

- The memory cell C_t is very similar to the candidate hidden state in GRU, but now it is protected by its own forget and input gates.

4 Deep, bidirectional, and sequence-to-sequence models

4.1 Deep RNN/LSTM/GRU

- A single recurrent layer may be too simple. Stacking layers increases capacity: lower layers learn shorter-term dynamics, higher layers learn high-level trends.

- The l -th hidden layer follows $\mathbf{H}_t^{(l)} = \varphi_l(\mathbf{H}_t^{(l-1)} \mathbf{W}_{xh}^{(l)} + \mathbf{H}_{t-1}^{(l)} \mathbf{W}_{hh}^{(l)} + \mathbf{b}_h^{(l)})$, with the output read from the last layer, $O_t = \sigma(\mathbf{H}_t^{(L)} \mathbf{W}_{ho} + b_o)$.

4.2 Bidirectional model

- Downstream information can be helpful: “I am ___ hungry, and I can eat half a pig” versus “I am ___ hungry, because I have already eaten half a pig.”

- A bidirectional model runs two recursions, forward over $\mathbf{H}_{t-1}^{(f)}$ and backward over $\mathbf{H}_{t+1}^{(b)}$, and concatenates both for the output $O_t = \sigma(\text{cat}(\mathbf{H}_t^{(f)}, \mathbf{H}_t^{(b)}) \mathbf{W}_{ho} + b_o)$.

- Limitations: it cannot do forward prediction (training sees both directions, testing only one, giving poor performance). It fits filling in missing words, annotating tokens such as named-entity recognition, and sequence encoding for machine translation.

- It is very, very slow: the forward pass requires both a forward and a backward recursion.

4.3 Sequence-to-sequence model

- Machine translation maps a sequence from one language to another, e.g. “Thank you” → “xie xie ni”, usually with word-level tokenization.

- Sequences have different lengths, so a direct position-wise mapping does not exist. An encoder transforms the input into a state of fixed shape; a decoder maps that state to a variable-length sequence.

- Encoder: $h_t = f(x_t, h_{t-1})$ and the context $c = q(h_1, \ldots, h_T)$, for example $c = h_T$. Decoder: $s_{t'} = g(y_{t'}, c, s_{t'-1})$.

- Loss function: each output position is a classification, so the training loss is cross-entropy per position. Prediction stops when “<eos>” is generated.

- Evaluation: BLEU (Bilingual Evaluation Understudy). p_n is the precision of n-grams, the number of matched n-grams divided by the number of n-grams in the predicted sequence.

- Example: target ABCDEF, predicted ABBCD, so $p_1 = 4/5$, $p_2 = 3/4$, $p_3 = 1/3$, $p_4 = 0$. BLEU multiplies the n-gram precisions with a brevity penalty; the higher, the better.

$$\text{BLEU} = \exp\left(\min\left(0, 1 - \frac{\text{len target}_s}{\text{len pred}_s}\right)\right) \cdot \prod_{n=1}^{k} p_n^{1/2^n}$$

4.4 Search strategies

- Greedy search picks the most probable token at each step, but a locally best choice can miss a globally better path: 0.5·0.4·0.4·0.6 = 0.048 versus 0.5·0.3·0.6·0.6 = 0.054.

- Exhaustive search over $|D|$ tokens and L steps costs $|D|^L$ — infeasible for e.g. $|D| = 10000$, $L = 10$.

- Beam search: keep k partial sequences, expand each by the top candidates, and select the k best among the k|D| possibilities at every step.

5 Attention

- In seq2seq the hidden state is very important, yet there is only one hidden state aggregating the sentence; it cannot guarantee a proper representation for each word. The natural idea: encode each word by considering its context.

• Representation with neighbor information: an average $f(x) = (1/n)\Sigma y_i$, or a weighted average $f(x) = \Sigma \alpha(x, x_i) y_i$ where α reflects similarity or distance. In the most general form this is attention.

5.1 Attention for machine translation

• Hidden state of each word: consider interactions with all other words directly. A query hidden state gathers the states of the tokens through a weighted average, $f(x) = \Sigma \alpha(x, x_i) y_i$.

• Three questions: how to encode characters (keys)? how to define the values? how to define the attention weight function?

• Character encoding (keys): one-hot encoding, or a fully connected network applied to the one-hot vector. Values: a fully connected network over the one-hot vector.

• Attention weights (dot-product): $\alpha(q, k_i) = exp(a(q, k_i)) / \Sigma j_j \exp(a(q, k_j))$ with $a(q, k) = q \, \mathbf{T} / \sqrt{d}$.

• Loss function: cross-entropy for each word prediction. Training: the common optimizers with back-propagation.

5.2 Self-attention

• For each input x_i in the sentence: $y_i = \Sigma j_j \alpha(x_i, x_j) v_j$.

• Each output y_i is a hidden state for downstream tasks, attending against the sentence itself — including the token itself.

• It can be computed in parallel for all tokens; encoding and embedding are the most important things for NLP.

5.3 Position encoding

• Attention pools over the sequence without any order information. Time series, sequences, and text all carry order, so position must be injected.

• For $X \in \mathbb{R}^{n \times d}$, a position matrix $P \in \mathbb{R}^{n \times d}$ adds sine and cosine features of the row index, with frequencies that decay along the dimension:

$$p_{i,2j} = \sin\left(\frac{i}{10000^{\frac{2j}{d}}}\right), \qquad p_{i,2j+1} = \cos\left(\frac{i}{10000^{\frac{2j}{d}}}\right).$$

- It looks like a weird function but people use it; it encodes both absolute and relative positional information. The input dimension equals the raw encoding dimension plus the position encoding dimension.

5.4 Multi-head attention and the Transformer

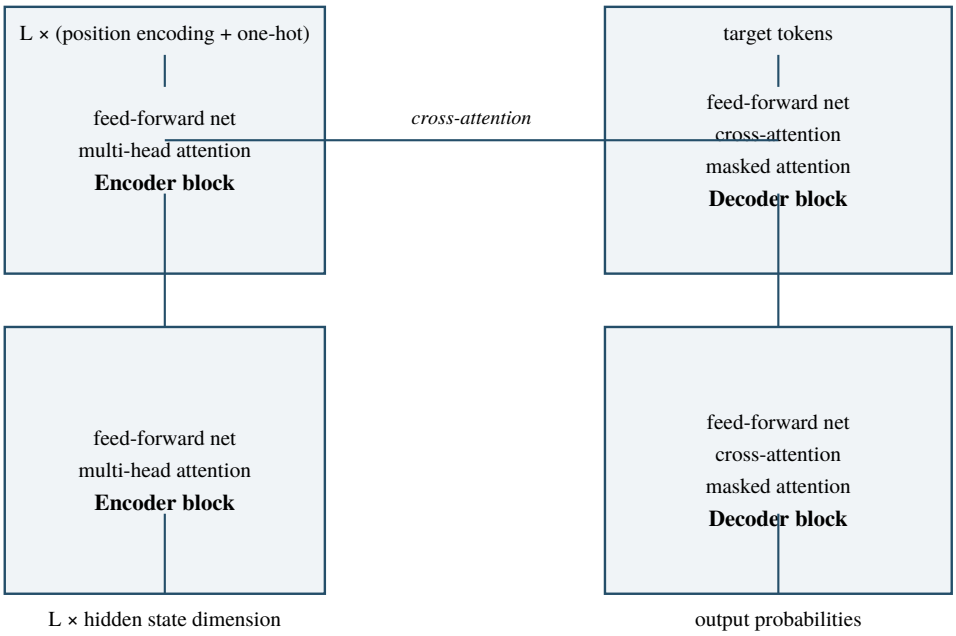


Figure 3. The Transformer stacks encoder and decoder blocks; each block combines multi-head attention with fully connected layers, and the decoder attends to the encoder outputs.

- Input of an attention layer: L × (position embedding + one-hot). Output: L × hidden-state dimension.

- To increase complexity: multi-head attention and stacking more attention layers.

- Multi-head attention projects queries, keys, and values into different representation subspaces, attends in each head, and concatenates the outputs (dimension × *n_head*).

- A Transformer block: multi-head attention learns the hidden state; fully connected layers control the dimension (for easy implementation); masked attention ignores padding positions. Stacking multiple blocks increases complexity.